Momentum Life AI Gateway - Hackathon Guide

Momentum Life's AI Gateway provides a unified interface to access AI models on AWS Bedrock. The gateway allows custom setup per app/user, which includes rate limiting, budgets, and model allowances. This document will explain the setup and some usage examples.

Team Setup

For each team we set a budget, as well as rate limits. This allows each team to have their own limits. This will be given to the teams on the day of the Hackathon. Each team will receive their own "virtual key", which will route custom requests through Momentum Life's AI Gateway. The limits include:

- Budget allowance per team for the entire hackathon (50 USD per team)
- Maximum tokens per timeframe (e.g., per minute) (will be given at the start of the hackathon)
- Maximum requests per timeframe (e.g., per minute) (will be given at the start of the hackathon)

Important: Budget Constraints

Please consult https://aws.amazon.com/bedrock/pricing/ for pricing guidelines (region: `eu-west-1`). As in real life, everything costs money and therefore the budget limits give realistic constraints to real-world use cases. First, experiment with partial documents and smaller models before scaling the use case!

You are more than welcome to deploy and use local models.

Available AWS Models

The following are available and tested:

- Chat models:
  - `bedrock/eu.amazon.nova-2-lite-v1:0`
  - `bedrock/eu.amazon.nova-lite-v1:0`
  - `bedrock/eu.amazon.nova-micro-v1:0`
  - `bedrock/eu.amazon.nova-pro-v1:0`
  - `bedrock/anthropic.claude-3-haiku-20240307-v1:0`
  - `bedrock/anthropic.claude-3-sonnet-20240229-v1:0`
  - `bedrock/eu.anthropic.claude-haiku-4-5-20251001-v1:0`
  - `bedrock/eu.anthropic.claude-sonnet-4-20250514-v1:0`
  - `bedrock/eu.anthropic.claude-sonnet-4-5-20250929-v1:0`

    - `bedrock/eu.anthropic.claude-sonnet-4-6`
    - `bedrock/google.gemma-3-12b-it`
    - `bedrock/google.gemma-3-27b-it`
    - `bedrock/google.gemma-3-4b-it`
    - `bedrock/minimax.minimax-m2`
    - `bedrock/minimax.minimax-m2.5`
    - `bedrock/minimax.minimax-m2.1`
    - `bedrock/mistral.devstral-2-123b`
    - `bedrock/mistral.magistral-small-2509`
    - `bedrock/mistral.ministral-3-14b-instruct`
    - `bedrock/mistral.ministral-3-3b-instruct`
    - `bedrock/mistral.ministral-3-8b-instruct`
    - `bedrock/mistral.mistral-7b-instruct-v0:2`
    - `bedrock/mistral.mistral-large-2402-v1:0`
    - `bedrock/mistral.mixtral-8x7b-instruct-v0:1`
    - `bedrock/mistral.voxtral-mini-3b-2507`
    - `bedrock/mistral.voxtral-small-24b-2507`
    - `bedrock/nvidia.nemotron-nano-12b-v2`
    - `bedrock/nvidia.nemotron-nano-3-30b`
    - `bedrock/nvidia.nemotron-nano-9b-v2`
    - `bedrock/nvidia.nemotron-super-3-120b`
    - `bedrock/openai.gpt-oss-safeguard-120b`
    - `bedrock/openai.gpt-oss-safeguard-20b`
    - `bedrock/qwen.qwen3-32b-v1:0`
    - `bedrock/qwen.qwen3-coder-30b-a3b-v1:0`
    - `bedrock/qwen.qwen3-next-80b-a3b`
    - `bedrock/qwen.qwen3-vl-235b-a22b`
    - `bedrock/zai.glm-4.7-flash`
- Embedding models:
    - `bedrock/amazon.titan-embed-image-v1`
    - `bedrock/amazon.titan-embed-text-v2:0`

Usage Examples

```
URL = "https://riskbifrostent.momentum.co.za/openai" # to use OpenAI
V_KEY = "sk-bf-..." # virtual key
```

OpenAI SDK

```
from openai import OpenAI

client = OpenAI(
     base_url=URL,
     default_headers={
          "x-git": "risk-ai-api",
          "x-bf-vk": V_KEY,
     }
)
```

Chat Completion

```
response = client.chat.completions.create(
     model="bedrock/anthropic.claude-3-haiku-20240307-v1:0",
     messages=[{"role": "user", "content": "Hi"}]
)
```

Structured Output

```
from pydantic import BaseModel

class CalendarEvent(BaseModel):
     name: str
     date: str
     participants: list[str]

response = client.responses.parse(
     model="bedrock/anthropic.claude-3-haiku-20240307-v1:0",
     input=[
          {"role": "system", "content": "Extract the event information."},
          {"role": "user", "content": "Alice and Bob are going to a science
fair on Friday.",},
     ],
     text_format=CalendarEvent,
)

print(response)
event = response.output_parsed
print(event)
```

Embeddings

```
embedding = client.embeddings.create(
     model="bedrock/amazon.titan-embed-text-v2:0",
     input=["The food was delicious and the waiter...", "The movie was
terrible and the plot was..."],
```

```
    encoding_format="float"
)

print(embedding)
```

Streaming

```
stream = client.responses.create(
    model="bedrock/anthropic.claude-3-haiku-20240307-v1:0",
    input=[{
        "role": "user",
        "content": "Say 'double bubble bath' ten times fast.",
    }],
    stream=True,
)

for event in stream:
    print(event)
```

Please note: using the OpenAI SDK is just an example. Any OpenAI-compatible SDK/API should work. More information regarding the OpenAI SDK integration here: https://docs.getbifrost.ai/integrations/openai-sdk/overview
For other providers, please consult:

- Bedrock: https://docs.getbifrost.ai/integrations/bedrock-sdk/overview
- GenAI: https://docs.getbifrost.ai/integrations/genai-sdk/overview
- LiteLLM: https://docs.getbifrost.ai/integrations/litellm-sdk
- LangChain: https://docs.getbifrost.ai/integrations/langchain-sdk
- PydanticAI: https://docs.getbifrost.ai/integrations/pydanticai-sdk

Additional Recommendations

It is known that Bedrock does fail at times. We've built retry loops in the AI gateway, but even then it is not guaranteed to succeed. It is recommended that you build your own retry loops (exponential backoff, etc.) to prevent disappointment.